

Sprint 3 & 4 Implementation Guide

Sprint 3: Persistence Model Implementation - Complete Summary

✓ What Was Implemented

1. JPA Entities with @Inheritance

Notification Hierarchy (`Notification.java`)

- Base abstract class with `@Inheritance(strategy = InheritanceType.JOINED)`
- Creates separate tables for each subclass with foreign key relationships
- Three concrete implementations:
 - `DocumentNotification` - for document-related notifications
 - `ReleaseNotification` - for software release notifications
 - `SystemNotification` - for system-wide notifications
- Demonstrates polymorphic queries and type-specific fields

Key Concepts:

- `@DiscriminatorColumn` - identifies the type of entity
- `@DiscriminatorValue` - specifies the discriminator value for each subclass
- Joined table inheritance creates normalized database structure

2. Value Objects

DocumentAuditInfo (`DocumentAuditInfo.java`)

- `@Embeddable` value object for audit tracking
- Contains: creation timestamp, last modified timestamp, modifier, version count
- Embedded in Document entity with `@Embedded`
- Demonstrates immutable data patterns and composition

Key Concepts:

- Value objects have no identity of their own
- Always part of an aggregate root
- Implement equals/hashCode based on value, not identity

3. Custom Converters with @Convert

DocumentPriorityConverter (`DocumentPriorityConverter.java`)

- Implements `AttributeConverter<DocumentPriority, String>`
- Converts enum to custom database codes (L, M, H, C)
- `@Converter(autoApply = true)` - automatically applies to all fields of that type
- Demonstrates custom database mappings beyond default JPA behavior

Key Concepts:

- Converters allow custom mapping between Java types and database columns
- Useful for legacy databases or optimized storage formats
- Can be applied globally or per-field basis

4. Complete Aggregate Relationships

Updated Document Entity (`Document.java`)

- `@OneToMany` with full aggregate configuration:
 - `cascade = CascadeType.ALL` - all operations cascade to children
 - `orphanRemoval = true` - removes orphaned children automatically
 - `fetch = FetchType.LAZY` - loads children only when accessed
 - `mappedBy = "document"` - defines bidirectional relationship owner

Key Concepts:

- **Cascade:** Operations on parent affect children (persist, merge, remove, etc.)
- **Orphan Removal:** If child is removed from collection, it's deleted from database
- **Fetch Strategy:** LAZY = on-demand loading, EAGER = immediate loading
- **Bidirectional Mapping:** One side is owner (with `@JoinColumn`), other uses `mappedBy`

Aggregate Helper Methods:

```
java
```

```
public void addFeatureLink(DocumentFeatureLink link) {  
    featureLinks.add(link);  
    link.setDocument(this); // Maintain bidirectional consistency  
}  
  
public void removeFeatureLink(DocumentFeatureLink link) {  
    featureLinks.remove(link);  
    link.setDocument(null); // Break relationship  
}
```

5. Entity Listeners (@EntityListeners)

DocumentAuditListener (DocumentAuditListener.java)

- `@PrePersist` - executes before entity is first saved
 - Sets timestamps, default values, computes file size
- `@PreUpdate` - executes before entity is updated
 - Updates modification timestamp, recomputes derived values
- `@PreRemove` - executes before entity is deleted
 - Logs deletion, can trigger cleanup operations
- `@PostLoad` - executes after entity is loaded from database
 - Validates integrity, can initialize transient fields

Key Concepts:

- **Lifecycle Callbacks:** Hooks into JPA entity lifecycle
- **Computation Rules:** Automatically calculate derived values
- **Audit Trails:** Track who/when changes were made
- **Business Rules:** Enforce constraints and validations

Example Use Cases:

- Set creation timestamps automatically
- Calculate file sizes from file paths
- Validate data integrity on load
- Archive deleted records
- Send notifications on status changes

6. Enhanced Repository Queries

Spring Data JPA Methods (`DocumentRepositoryEnhanced.java`)

Derived Query Methods:

```
java

List<Document> findByStatus(DocumentStatus status);
List<Document> findByTitleContainingIgnoreCase(String titlePart);
List<Document> findByUploader_UserId(int uploaderId);
long countBySoftwareRelease_ReleaseId(int releaseId);
```

Custom @Query with JPQL:

```
java

@Query("SELECT d FROM Document d " +
      "LEFT JOIN FETCH d.uploader " +
      "WHERE d.softwareRelease.releaseId = :releaseId")
List<Document> findByReleaseIdWithDetails(@Param("releaseId") int releaseId);
```

Native SQL Queries:

```
java

@Query(value = "SELECT u.UserID, COUNT(d.DocumentID) as doc_count " +
      "FROM User_app u JOIN Document d ON u.UserID = d.UploaderID " +
      "GROUP BY u.UserID LIMIT :limit",
      nativeQuery = true)
List<Object[]> findTopUploaders(@Param("limit") int limit);
```

Modifying Queries:

```
java

@Modifying
@Query("UPDATE Document d SET d.status = :newStatus " +
      "WHERE d.softwareRelease.releaseId = :releaseId")
int updateDocumentStatus(@Param("releaseId") int releaseId,
      @Param("newStatus") DocumentStatus newStatus);
```

Key Concepts:

- **Method Name Derivation:** Spring generates queries from method names
- **JPQL:** Object-oriented query language (queries entities, not tables)
- **Native SQL:** Direct database SQL when JPQL is insufficient
- **JOIN FETCH:** Solves N+1 query problem by eager loading in one query
- **@Modifying:** Required for UPDATE/DELETE queries
- Must use with `@Transactional` for modifying queries

7. Generic Repository Pattern (Bonus)

Repository Interface (`Repository.java`)

- Defines standard CRUD operations
- Generic type parameters: `<T, ID>`
- Contract for all repository implementations

RepositoryJPA Implementation (`RepositoryJPA.java`)

- Abstract base class using `EntityManager`
- `@PersistenceContext` injects JPA EntityManager
- Implements all CRUD operations generically
- Subclasses provide entity-specific operations

DocumentRepositoryJPA Concrete Implementation

- Extends `RepositoryJPA<Document, Integer>`
- Adds custom queries specific to Document
- Uses `getEntityManager()` for direct JPA operations

Key Concepts:

- **Generic Programming:** Write once, use for all entities
- **EntityManager:** Low-level JPA API for database operations
- **Template Method Pattern:** Base class provides structure, subclasses fill details
- **Type Safety:** Compile-time checking of entity types

When to Use:

- When you need more control than Spring Data JPA provides
- Complex queries that benefit from dynamic query construction
- Custom transaction management
- Integration with other persistence frameworks

8. JUnit Test Classes

Repository Tests (`DocumentRepositoryTest.java`)

- `@SpringBootTest` - loads full Spring context
- `@Transactional` - each test runs in transaction, rolled back after
- Tests CRUD operations, custom queries, relationships

Service Tests (`DocumentWorkflowServiceTest.java`)

- Tests business logic layer
- Validates validation rules
- Tests exception handling

Entity Listener Tests (`DocumentAuditListenerTest.java`)

- Verifies lifecycle callbacks execute correctly
- Tests audit fields are set automatically
- Validates computation rules

Aggregate Tests (`AggregateRelationshipTest.java`)

- Tests cascade operations
- Verifies orphan removal works
- Validates bidirectional relationship consistency

Key Testing Concepts:

- **Integration Tests:** Test full stack including database
- **@Transactional:** Ensures test isolation, no data pollution
- **@BeforeEach:** Setup before each test
- **Assertions:** Verify expected behavior

- **Test Data:** Create minimal data needed for tests
-

Sprint 4: Building Web Services

REST API Implementation

1. RESTful Endpoints Structure

HTTP Methods and Their Meanings:

- `GET` - Retrieve resources (read-only, idempotent)
- `POST` - Create new resources (not idempotent)
- `PUT` - Update entire resource (idempotent)
- `PATCH` - Partial update (idempotent)
- `DELETE` - Remove resource (idempotent)

Status Codes:

- `200 OK` - Successful GET, PUT, PATCH
- `201 Created` - Successful POST
- `204 No Content` - Successful DELETE
- `400 Bad Request` - Invalid data
- `404 Not Found` - Resource doesn't exist
- `500 Internal Server Error` - Server-side error

2. DocumentRestController

Annotations:

- `@RestController` - Combines `@Controller` + `@ResponseBody`
- `@RequestMapping("/api/documents")` - Base URL for all endpoints
- `@CrossOrigin` - Enable CORS for cross-domain requests
- `@GetMapping`, `@PostMapping`, etc. - HTTP method mappings
- `@PathVariable` - Extract from URL path: `/api/documents/{id}`
- `@RequestParam` - Extract from query string: `/api/documents?status=DRAFT`
- `@RequestBody` - Parse JSON from request body

Key Endpoints:

java

```
GET /api/documents - Get all documents
GET /api/documents/{id} - Get specific document
GET /api/documents/product/{id} - Get documents by product
POST /api/documents - Create document
PUT /api/documents/{id} - Update document
PATCH /api/documents/{id}/status - Update status only
DELETE /api/documents/{id} - Delete document
GET /api/documents/search - Search with filters
GET /api/documents/count - Get total count
```

ResponseEntity:

- Wraps response data with HTTP status
- `ResponseEntity.ok(data)` - 200 with body
- `ResponseEntity.notFound().build()` - 404 no body
- `ResponseEntity.status(HttpStatus.CREATED).body(data)` - 201 with body

3. Data Transfer Objects (DTOs)

Why Use DTOs:

- Decouple API from internal domain model
- Control what data is exposed
- Avoid circular references in JSON serialization
- Version API independently from domain

Record Classes (Java 16+):

java


```
record DocumentCreateRequest(  
    String title,  
    String filePath,  
    String version,  
    int uploaderId,  
    int releaseId,  
    int typeId,  
    DocumentStatus status  
) {}
```

Benefits:

- Immutable by default
- Automatic constructor, getters, equals, hashCode, toString
- Compact syntax for data carriers
- Perfect for DTOs

4. Exception Handling

@RestControllerAdvice - Global exception handler for all controllers

@ExceptionHandler - Methods to handle specific exceptions

ErrorResponse DTO:

```
java  
  
record ErrorResponse(  
    int status,    // HTTP status code  
    String error,  // Error category  
    String message, // Detailed message  
    String path,   // Request path  
    LocalDateTime timestamp  
) {}
```

Common Patterns:

```
@ExceptionHandler(EntityNotFoundException.class)
public ResponseEntity<ErrorResponse> handleNotFound(...) {
    return ResponseEntity.status(404).body(errorResponse);
}
```

Benefits:

- Consistent error format across API
- Single place to handle exceptions
- Proper HTTP status codes
- Detailed error information for debugging

5. API Testing

@AutoConfigureMockMvc - Enables MockMvc for testing

MockMvc:

- Simulates HTTP requests without starting server
- Fast integration testing
- Full Spring context

Example Test:

```
java
mockMvc.perform(get("/api/documents/1"))
    .andExpect(status().isOk())
    .andExpect(jsonPath("$.documentId").value(1))
    .andExpect(jsonPath("$.title").exists());
```

Testing Patterns:

- Test all HTTP methods (GET, POST, PUT, DELETE)
 - Test success cases (200, 201, 204)
 - Test error cases (400, 404, 500)
 - Verify JSON structure with JsonPath
 - Test request/response body serialization
-

Key Takeaways

Sprint 3 Achievements:

- ✓ Complete JPA entity model with inheritance
- ✓ Value objects and custom converters
- ✓ Full aggregate configuration with cascade/orphan removal
- ✓ Entity listeners for automatic computation and audit
- ✓ Enhanced repositories with custom queries
- ✓ Generic repository pattern (bonus)
- ✓ Comprehensive JUnit tests

Sprint 4 Achievements:

- ✓ RESTful API controllers for all entities
- ✓ Proper HTTP methods and status codes
- ✓ DTO pattern for request/response
- ✓ Global exception handling
- ✓ Complete API integration tests

Best Practices Learned:

1. **Lazy Loading:** Default to LAZY fetch to avoid performance issues
2. **Bidirectional Relationships:** Always maintain both sides
3. **Cascade Carefully:** Don't cascade delete unless intentional
4. **Use DTOs:** Never expose entities directly in REST APIs
5. **Test Thoroughly:** Integration tests catch issues early
6. **Error Handling:** Provide meaningful error messages
7. **Entity Listeners:** Automate repetitive tasks
8. **Query Optimization:** Use JOIN FETCH to avoid N+1 queries

Common Pitfalls to Avoid:

- ✗ Using EAGER fetch everywhere (performance killer)
 - ✗ Not using @Transactional for modifying queries
 - ✗ Circular references in JSON (use DTOs)
 - ✗ Not handling exceptions (returns 500 for everything)
 - ✗ Forgetting to maintain bidirectional relationships
 - ✗ Not testing with realistic data scenarios
 - ✗ Exposing internal exceptions to API clients
-

Next Steps

1. **Add Authentication/Authorization** (Spring Security)
2. **API Documentation** (Swagger/OpenAPI)
3. **Pagination** for large result sets
4. **Filtering and Sorting** enhancements
5. **File Upload** for documents
6. **Async Processing** for long operations
7. **Caching** for frequently accessed data
8. **API Versioning** strategy

Database Setup Reminder

Don't forget to update your database schema with:

- New Notification tables for inheritance example
- DocumentSize column in Document table
- Priority column in Document table
- DocumentAuditInfo fields (CreatedAt, LastModifiedAt, LastModifiedBy, VersionCount)

SQL migration scripts should be created for production deployments!